

Extraction d'informations dans les décisions de justice : règles, TF-IDF, réseau convolutif et CamemBERT sur les arrêts de la Cour de cassation

Wissem Sahli

Projet de traitement automatique du langage naturel (NLP) — octobre 2026

Résumé

Les décisions de justice sont des textes longs, peu structurés et rédigés dans une langue spécialisée. Nous proposons une chaîne de traitement qui transforme un arrêt de la Cour de cassation, fourni en PDF natif ou scanné, en un document JSON structuré : métadonnées (chambre, date, numéro de pourvoi, président), récit (faits, moyens), fondements juridiques (textes de loi cités, décision attaquée) et issue (verdict, ordres, motifs). Les champs semi-structurés sont extraits par des règles (expressions régulières), évaluées sur 1 959 décisions annotées ; la date est retrouvée dans 93 % des cas grâce à une règle qui lit les dates écrites en toutes lettres. L'issue est prédite à partir des seules *motivations* par trois modèles comparés sur les mêmes 10 000 décisions de test : (A) prétraitement spaCy, TF-IDF et régression logistique, (B) réseau convolutif Keras entraîné de zéro, (C) CamemBERT fine-tuné. Sur un corpus de 100 000 arrêts tirés des 553 075 décisions de Judilibre, les F1 macro sont respectivement de 0,964, 0,959 et 0,955 (CamemBERT n'ayant vu que 20 000 exemples, faute de puissance de calcul). Testée sur les arrêts réels de 2026 (95 arrêts évaluable sur 104), postérieurs aux données d'entraînement, la chaîne PDF → JSON retrouve l'issue avec une accuracy de 100 % par règles sur le dispositif et de 97 % pour le meilleur modèle lisant les motivations.

1 Introduction et problématique

Depuis la mise en place de l'open data des décisions de justice, la Cour de cassation diffuse ses arrêts sur la plateforme **Judilibre** : plus d'un demi-million de décisions pseudonymisées sont accessibles librement. Ce volume dépasse de loin ce qu'un juriste peut lire. Or l'information utile d'un arrêt tient en quelques champs : qui a jugé, quand, sur quels textes de loi, et avec quel résultat. La retrouver demande pourtant de lire un texte de plusieurs pages, écrit dans un style très codifié.

Choix du sujet. Ce sujet réunit trois difficultés classiques du NLP : des **textes longs** (649 mots en médiane, jusqu'à 183 000), un **vocabulaire spécialisé** (« pourvoi », « dispositif », « moyen annexé ») et des **classes déséquilibrées** (54 % de rejets). Il a aussi une utilité concrète (legal tech, recherche juridique) et dispose de données ouvertes de grande taille, ce qui permet une évaluation sérieuse.

Problématique. *Comment extraire automatiquement et de manière fiable les informations clés d'un arrêt, et quelle approche — règles, modèle statistique classique, réseau de neurones entraîné de zéro ou modèle de langue pré-entraîné — offre le meilleur compromis entre performance, coût de calcul et interprétabilité ?*

Extraction et non prédiction. Nous ne cherchons pas à deviner l'issue d'un procès futur : nous lisons une décision déjà rendue pour en extraire l'information. Le dispositif écrivant la réponse en toutes lettres (« CASSE ET ANNULE », « REJETTE le pourvoi »), les modèles d'apprentissage reçoivent uniquement les *motivations*, c'est-à-dire le raisonnement de la Cour, afin de mesurer s'ils comprennent ce raisonnement.

Contributions.

- une chaîne complète PDF → JSON qui détecte si le PDF est natif (PyMuPDF) ou scanné (OCR Tesseract) ;
- des règles d'extraction évaluées sur 1 959 décisions, dont une règle qui convertit les dates écrites en toutes lettres ;
- une comparaison de trois modèles (TF-IDF, CNN Keras, CamemBERT) et d'une règle, à données égales et inégales ;
- un test de généralisation sur des arrêts réels de 2026 (95 évaluable sur 104 extraits), postérieurs à toutes les données d'entraînement.

2 État de l'art

2.1 NLP juridique et analyse de décisions

Zhong et al. [6] dressent un panorama de l'intelligence artificielle juridique : recherche d'information, extraction d'éléments, prédiction de jugements. Aletras et al. [7] classent les arrêts de la Cour européenne des droits de l'homme (violation ou non) avec des n-grammes et un SVM (79 % d'accuracy). Chalkidis et al. [8] reprennent cette tâche avec des réseaux de neurones et BERT. Pour le français, Şulea et al. [9] prédisent le domaine et la décision d'arrêts de la Cour de cassation avec un SVM linéaire et obtiennent des scores très élevés. Medvedeva et al. [10] soulignent cependant que la plupart de ces travaux utilisent des textes rédigés *après* la décision : ils identifient l'issue plus qu'ils ne la prédisent. Notre étude assume ce cadre d'**extraction**.

2.2 Représentations et modèles de texte

La pondération **TF-IDF** [12] associée à un classifieur linéaire reste une référence solide en classification de documents. Les réseaux convolutifs appliqués aux plongements de mots (Kim [11]) capturent des motifs locaux de quelques mots. L'architecture **Transformer** [1], fondée uniquement sur l'attention, a conduit aux modèles pré-entraînés comme BERT [2], puis à sa version française

CamemBERT [3]. Dans le domaine juridique, **LEGAL-BERT** [4] est pré-entraîné sur des textes juridiques anglais et évalué sur le banc d'essai LexGLUE [5] ; **JuriBERT** [13] adapte ce principe au droit français. La limite de 512 tokens de ces modèles pose problème pour les textes longs, ce que des architectures comme Longformer [21] cherchent à résoudre.

2.3 Extraction par règles et entités nommées

Les métadonnées juridiques (dates, numéros, juridictions) suivent des formats stables : les expressions régulières restent efficaces et interprétables pour les extraire. La reconnaissance d'entités nommées juridiques (juges, parties, lois) par apprentissage demande en revanche un corpus annoté [14].

Positionnement. Peu de travaux comparent, sur des arrêts français et sous contrainte de calcul, des règles, un modèle classique, un réseau entraîné de zéro et un modèle pré-entraîné, en testant la chaîne complète sur de vrais PDF postérieurs aux données d'entraînement. C'est ce que nous proposons.

3 Données

3.1 Choix du jeu de données

Trois pistes ont été étudiées. (1) La **jurisprudence tunisienne** : la base du ministère de la Justice publie environ 12 000 arrêts de principe de la Cour de cassation, mais uniquement en arabe, sans jeu de données prêt pour le NLP ni annotations ; tout serait à construire. (2) Des corpus **anglophones** (Australie, CEDH) : bien outillés, mais hors du cadre francophone du projet. (3) **Judilibre**, l'open data de la Cour de cassation, redistribué sur Hugging Face [18] : 553 075 arrêts en français, sous Licence Ouverte Etalab 2.0, avec des champs structurés (date, chambre, issue, découpage en zones, textes visés) qui servent de **vérité terrain gratuite**. Nous retenons ce troisième choix.

3.2 Description et analyse exploratoire

Chaque ligne du fichier (JSONL compressé, 1 Go) contient le texte intégral et 33 champs. Le corpus totalise **639 millions de mots** ; 99 % des décisions sont postérieures à 1970 (figure 1).

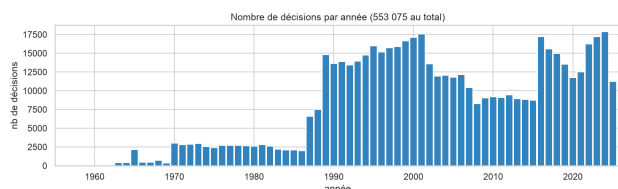


Figure 1 : Nombre de décisions par année (553 075 arrêts de la Cour de cassation).

Les 14 issues d'origine sont très déséquilibrées : *Rejet* (54,2 %) et *Cassation* (30,7 %) dominent. Nous les regroupons en **4 classes** : Rejet, Cassation, Irrecevabilité (4,0 %) et Autre (déchéance, non-lieu, QPC, annulation..., 11,1 %) (figure 2).

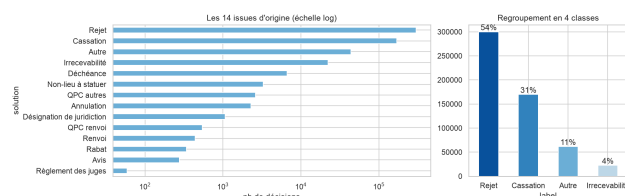


Figure 2 : Issues d'origine (échelle logarithmique) et regroupement en 4 classes.

Les textes sont longs : médiane de 649 mots, et **75,5 %** des décisions dépassent la limite de 512 tokens de CamemBERT (figure 3). Seules 65 % des décisions possèdent un découpage en zones ; parmi elles, les motivations et le dispositif sont presque toujours présents (99 %), alors que l'exposé des faits (34 %) et les moyens (68 %) manquent souvent. L'issue dépend aussi de la chambre : les chambres civiles cassent dans 31 à 37 % des cas, la chambre criminelle dans 19 % seulement, avec 24 % d'issues « Autre ».

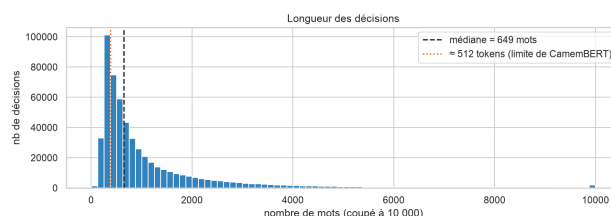


Figure 3 : Longueur des décisions en mots ; la ligne pointillée orange indique la limite approximative de CamemBERT.

3.3 Construction des corpus

Le script `prepare_data.py` lit le fichier ligne par ligne, sans le charger en mémoire, et produit : (i) une ligne de statistiques par décision (553 075, pour l'analyse exploratoire) ; (ii) un **corpus de modélisation de 100 000 décisions**, tirées au hasard (graine 42) parmi celles qui ont des motivations et un dispositif ; (iii) un **jeu d'évaluation des règles** de 1 959 décisions avec leur texte intégral. Le tableau 1 montre que le corpus de modélisation reproduit la distribution des décisions ayant des zones.

Tableau 1 : Distribution des classes selon le périmètre.

Classe	Base complète	Base avec zones	Corpus 100k
Rejet	54,2 %	55,4 %	55,4 %
Cassation	30,7 %	25,6 %	25,6 %
Irrecevabilité	4,0 %	4,8 %	4,9 %
Autre	11,1 %	14,2 %	14,1 %

Dans les motivations, les mots les plus caractéristiques de chaque classe ont un sens juridique net : « violé », « texte susvisé », « cassation encourue » pour la cassation ; « manifestement », « moyens annexés » pour les rejets non spécialement motivés ; « irrecevable », « articles 606 à 608 » pour l'irrecevabilité. Les motivations sont courtes (médiane de **146 mots**), ce qui en fait une entrée adaptée aux modèles.

3.4 PDF réels pour le test en production

Trois numéros du *Bulletin* de la Cour de cassation (civil de mai 2026, criminel de juin et juillet 2026) servent de test final : ils contiennent **104 arrêts distincts**, tous postérieurs à la dernière décision du jeu d'entraînement (mars 2025).

L'issue y est imprimée sous l'en-tête de chaque arrêt (« – Cassation – »), ce qui fournit la bonne réponse.

4 Méthodologie

4.1 Vue d'ensemble

La chaîne de traitement comporte cinq étapes :

- **Lecture** : si PyMuPDF trouve plus de 100 caractères par page sur les premières pages, le PDF est natif et son texte est lu directement ; sinon chaque page est convertie en image (300 dpi) puis reconnue par Tesseract en français [20] ;
- **Nettoyage** : suppression des en-têtes et numéros de page, des espaces insécables, recollage des mots coupés en fin de ligne ;
- **Découpage** : un Bulletin est séparé en arrêts grâce à leur en-tête (« Crim., 29 juillet 2026, n° 26-83.088 ») ; chaque arrêt est découpé en zones (introduction, exposé, moyens, motivations, dispositif) ;
- **Extraction par règles** des métadonnées, des textes de loi, de la décision attaquée et de l'issue ;
- **Classification** de l'issue à partir des motivations par un modèle d'apprentissage, puis écriture du JSON.

4.2 Extraction par règles

Deux styles de rédaction coexistent : le style ancien (« Attendu que... ; D'où il suit que le moyen n'est pas fondé ») et le *style direct* adopté depuis 2019 (« Faits et procédure », « Examen du moyen », « Réponse de la Cour »). Les marqueurs des zones couvrent les deux styles. Les principales règles sont :

- **date** : on relève toutes les dates d'audience, en chiffres ou **en toutes lettres** (« du neuf janvier deux mille trois »), converties par une petite grammaire des nombres français (« quatre vingt » = 80), et l'on garde la *dernière*, qui correspond au prononcé (la première peut être celle des débats) ;
- **chambre et numéro de pourvoi** (motif « 14-27.033 ») ;
- **textes de loi** : « article(s) X du code Y », avec une liste de 40 codes pour reconnaître les noms complets (« code de procédure pénale » et non « code de procédure ») ;
- **issue** : verbes du dispositif (CASSE → Cassation, REJETTE → Rejet, IRRECEVABLE → Irrecevabilité, sinon Autre) ;
- **ordres** : le dispositif est découpé en injonctions (« RENVOIE... », « Condamne... aux dépens »).

4.3 Prétraitement NLP avec spaCy

Pour les modèles A et B, les motivations passent par le modèle français `fr_core_news_sm` de spaCy [15] : **tokenisation**, **lemmatisation** (« violé », « viole » → « violer »), suppression des mots vides, de la ponctuation et des nombres. Les **négations** (« ne », « pas », « non », « sans »...) sont volontairement conservées : « le moyen *n'est pas* fondé » (rejet) ne doit pas devenir « moyen fondé ». Exemple : « D'où il suit que le moyen n'est pas fondé » → *suivre moyen ne pas fonder*.

4.4 Modèles de classification

Modèle A — TF-IDF + régression logistique. Unigrammes et bigrammes de lemmes (fréquence minimale 3, 100 000 variables, TF sous-linéaire), puis régression logistique avec pondération équilibrée des classes ; la régularisation $C \in \{0,5 ; 2 ; 8\}$ est choisie sur la validation. Un Naive Bayes multinomial sert de point de comparaison.

Modèle B — CNN Keras entraîné de zéro. Vocabulaire de 30 000 lemmes, séquences de 300 ; plongement de dimension 128 appris ; convolution 1D (128 filtres, fenêtre de 5 mots) ; max-pooling global ; dropout 0,3 ; couche dense de 64 ; softmax à 4 sorties ($\approx 3,9$ M de paramètres). Optimiseur Adam (taux 10^{-3}), lots de 128, arrêt précoce (patience 2, au plus 10 époques), poids de classes inversement proportionnels à leur fréquence.

Modèle C — CamemBERT fine-tuné. `camembert-base` [3] (110 M de paramètres) reçoit le texte brut, découpé en sous-mots. On garde les **256 derniers tokens** des motivations (troncature à gauche), là où la Cour conclut. Une couche de classification est ajoutée ; l'ensemble est entraîné 1 époque sur **20 000** décisions (taux $2 \cdot 10^{-5}$ décroissant linéairement jusqu'à 0, lots de 16, mêmes poids de classes).

5 Implémentation

Le projet est écrit en Python 3.12 et tourne sur un MacBook à puce **Apple M4 (16 Go)**. Le GPU est utilisé via **TensorFlow 2.18** et le greffon **tensorflow-metal** ; PyTorch n'est pas utilisé. Les principales bibliothèques sont spaCy 3.8, scikit-learn 1.9 [16], tf-keras 2.18, transformers 4.57 [17] (dernière version qui fournit les classes TensorFlow), PyMuPDF et Tesseract. Toutes les versions sont figées dans `requirements.txt` et la graine aléatoire est fixée à 42.

Tableau 2 : Organisation du code.

Fichier	Rôle
<code>scripts/prepare_data.py</code>	lecture du fichier brut, corpus et statistiques
<code>notebooks/01_EDA.ipynb</code>	analyse exploratoire (section 3)
<code>notebooks/02_NLP.ipynb</code>	règles, spaCy, modèles A/B/C, évaluation
<code>scripts/Txt_ToJSON.py</code>	PDF/texte → JSON (lecture, OCR, règles)
<code>scripts/predict.py</code>	JSON → issue prédite par le modèle choisi
<code>models/</code>	modèles entraînés et modèle final

Difficultés rencontrées. (1) `tensorflow-metal` n'est pas compatible avec TensorFlow 2.21 ni avec Python 3.14 : retour à TensorFlow 2.18 et Python 3.12. (2) Le nouvel optimiseur Adam de Keras fait planter le greffon Metal (erreur du « remapper » de graphe) : on utilise l'Adam *legacy*, recommandé par TensorFlow sur Apple Silicon. (3) CamemBERT est lent sur cette machine (≈ 4 exemples/s à l'entraînement), d'où la limite de 20 000 exemples. (4) Un Bulletin publie parfois le même arrêt sous deux thèmes : les doublons sont écartés par numéro de pourvoi.

6 Protocole expérimental

- **Découpage** stratifié du corpus de 100 000 décisions : 80 % entraînement, 10 % validation, 10 % test.
- **Choix des réglages et du modèle final** sur la validation uniquement ; le test n'est utilisé qu'une fois, pour le rapport.

- **Métriques** : accuracy, F1 par classe et surtout **F1 macro** (moyenne des F1 des 4 classes), insensible au déséquilibre ; temps d'entraînement et d'inférence par document.
- **Équité** : CamemBERT n'ayant vu que 20 000 exemples, A et B sont aussi ré-entraînés sur *les mêmes* 20 000 décisions.
- **Règles** : exact match par champ sur les 1 959 décisions du jeu d'évaluation, et rappel des textes du visa.
- **Généralisation** : chaîne complète (PDF → JSON → modèle) sur les arrêts des Bulletins 2026 dont l'issue est imprimée et les motivations extraites (95 sur 104).

7 Résultats

7.1 Extraction par règles

Tableau 3 : Extraction par règles : exact match sur les 1 959 décisions du jeu d'évaluation (sauf les deux dernières lignes : test de 10 000 décisions et Bulletins 2026).

Champ	Score	Détail par période (<2000 / 2000-14 / ≥2015)
Date de la décision	93,3 %	93 % / 98 % / 90 %
Chambre	96,2 %	100 % / 100 % / 91 %
N° de pourvoi	41,9 %	3 % / 2 % / 99 %
Cour d'appel attaquée	79,4 %	décisions où elle est connue
Textes du visa retrouvés	98,5 %	rappel
Issue (dispositif), test	96,3 %	F1 macro 0,933
Issue (dispositif), PDF 2026	100,0 %	95 arrêts

Les métadonnées sont très bien extraites quand elles sont écrites dans le texte. La règle des **dates en toutes lettres** est décisive : sans elle, aucune date antérieure à 2015 n'était retrouvée ; avec elle, 93 % et 98 % le sont. Le numéro de pourvoi n'est retrouvé que dans 42 % des cas, mais ce n'est pas un défaut de la règle : il n'est **écrit que dans 2,8 % des textes antérieurs à 2015**, alors qu'il est retrouvé dans 99 % des décisions récentes. Les dates manquées après 2015 concernent surtout les ordonnances de la Première présidence, qui n'ont pas d'en-tête daté. Enfin, la règle sur le dispositif donne l'issue avec 96,3 % d'accuracy ; elle est moins bonne sur « Irrecevabilité » (F1 0,860) et « Autre » (F1 0,909), classes aux formulations variées.

7.2 Classification de l'issue à partir des motivations

Tableau 4 : Classification de l'issue sur le test (10 000 décisions). Les trois dernières lignes sont entraînées sur les **mêmes 20 000 décisions**. En gras : meilleur modèle à 80k et meilleur modèle à 20k.

Modèle	n	Acc.	F1 macro	F1 Rej.	F1 Cass.	F1 Irr.	F1 Autre
Naive Bayes	80k	0,898	0,887	0,916	0,861	0,847	0,927
A · TF-IDF	80k	0,979	0,964	0,988	0,977	0,928	0,963
B · CNN	80k	0,973	0,959	0,984	0,967	0,924	0,961
A · TF-IDF	20k	0,970	0,953	0,982	0,965	0,912	0,955
B · CNN	20k	0,967	0,952	0,979	0,958	0,916	0,954
C · CamemBERT	20k	0,973	0,955	0,984	0,979	0,922	0,935

Les trois modèles dépassent largement Naive Bayes (F1 macro 0,887). Avec 80 000 exemples, le modèle le plus simple, **TF-IDF + régression logistique**, obtient le

meilleur F1 macro (0,964), devant le CNN (0,959). À **données égales** (20 000 exemples), l'ordre s'inverse : CamemBERT (0,955) devance la régression logistique (0,953) et le CNN (0,952). CamemBERT est le meilleur sur la classe Cassation (F1 0,979) mais le moins bon sur « Autre » (0,935). Les écarts entre les trois modèles restent faibles (moins d'un point de F1 macro). Le modèle final, choisi sur la **validation** (F1 macro 0,962), est donc A.

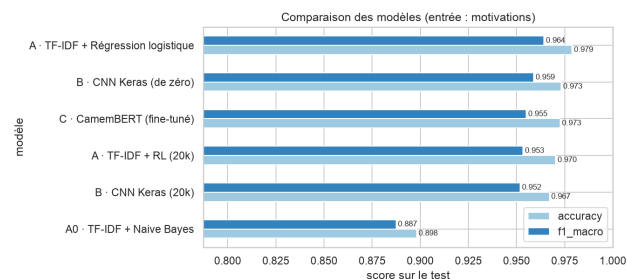


Figure 4 : Accuracy et F1 macro sur le test pour tous les modèles (entrée : motivations).

Tableau 5 : Coût de calcul sur le Mac M4. Pour A et B, il faut ajouter le prétraitement spaCy (≈ 8 ms par document sur 4 cœurs, 800 s pour 100 000 textes).

Modèle	Entraînement	Inférence / doc
Règles	—	0,01 ms
A · TF-IDF + RL (80k)	18 s	0,09 ms
B · CNN Keras (80k)	111 s	0,07 ms
C · CamemBERT (20k)	91 min	72 ms

7.3 Test en production sur les PDF de 2026

La chaîne complète a été appliquée aux 3 Bulletins : 95 arrêts possèdent à la fois une issue imprimée (49 cassations, 46 rejets) et des motivations exploitables.

Tableau 6 : Issue retrouvée sur les arrêts des Bulletins de mai à juillet 2026, postérieurs aux données d'entraînement.

Méthode	Accuracy	Erreurs
Règles (dispositif)	100,0 %	0
A · TF-IDF + RL	95,8 %	4
B · CNN Keras	88,4 %	11
C · CamemBERT	96,8 %	3

Sur ces décisions nouvelles, **CamemBERT est le meilleur modèle** (96,8 %), devant A (95,8 %), tandis que le CNN chute à 88,4 %. Seules deux classes sont présentes : l'accuracy est ici la bonne mesure, le F1 macro étant faussé dès qu'un modèle prédit une classe absente. Les 4 erreurs du modèle A sont toutes des rejets prédits « Cassation ».

8 Analyse critique

Un modèle simple suffit-il ? Sur le test, oui : le TF-IDF bat les réseaux, s'entraîne en 18 secondes et prédit environ 800 fois plus vite que CamemBERT hors prétraitement (encore 9 fois plus vite en comptant spaCy). La raison est visible dans les mots qu'il a appris (tableau 7) : les motivations contiennent des **formules quasi standardisées** (« violé le texte susvisé », « cassation encourue », « le moyen n'est pas fondé », « pourvoi irrecevable ») qu'un modèle de sacs de mots capte parfaitement. La tâche est donc plus proche de l'identification de l'issue que de sa prédiction, ce que

Medvedeva et al. [10] observent aussi. Les scores très élevés ne doivent pas être lus comme une « compréhension » fine du droit.

Tableau 7 : Lemmes et bigrammes les plus influents de la régression logistique (modèle A), par classe.

Rejet	Cassation	Irrecevabilité	Autre
moyen	violer texte	irrecevable	lieu
régulier forme	cassation encourir	pourvoi	convier
violer article	texte	recevabilité	sans objet
fonder	statuer	pas recevable	déchéance
moyen ne	violer	recevabilité pourvoi	moyen nature
pas fonder	fonder article	pourvoi irrecevable	cassation constater
régulier	susviser	fonder arrêt	pourvoi
légalement	texte susviser	irrecevable pourvoi	recours pièce

Où se situe l'avantage de CamemBERT ? À données égales, il est le meilleur, et c'est lui qui généralise le mieux aux arrêts de 2026, rédigés dans le style direct récent. Son pré-entraînement sur du français général l'aide à lire des formulations nouvelles, là où le CNN, qui a appris ses plongements de zéro, se trompe davantage. Son coût reste un frein : 91 minutes d'entraînement pour 20 000 exemples et 72 ms par décision. Avec 80 000 exemples et plusieurs époques, il dépasserait probablement les autres modèles ; nous n'avons pas pu le vérifier.

Analyse des erreurs. Les erreurs du modèle final se concentrent sur les décisions anciennes (accuracy 96,9 % avant 2000 contre 98,8 % pour le style direct). Les confusions les plus fréquentes sont « Autre » ↔ « Rejet » et « Cassation » → « Rejet » ou « Irrecevabilité ». L'examen des exemples montre trois causes : (i) des **négations ou tournures inhabituelles** : « la cour d'appel [...] n'a pu violer les textes précités » devient, une fois lemmatisé, « violer texte », typique d'une cassation ; (ii) des décisions procédurales classées « Autre » (rectification d'erreur matérielle, requête en interprétation) dont le texte parle de cassation ou de rejet ; (iii) des cassations sans renvoi, dont la fin des motivations porte sur l'application de la règle de droit (« il y a lieu de mettre fin au litige ») plutôt que sur la violation.

Règles ou modèles ? Quand le dispositif est disponible, la règle atteint 96,3 % sur le test et 100 % sur les Bulletins : elle reste la meilleure solution pour remplir le champ « verdict » du JSON. Les modèles sont utiles comme **contrôle de cohérence** (un désaccord entre la règle et le modèle signale un arrêt à vérifier) et quand le dispositif manque ou est mal extrait (OCR imparfait).

Raccourcis appris. L'analyse exploratoire a fait apparaître le nom d'une magistrate parmi les mots caractéristiques de la classe « Autre » (ordonnances signées par la même personne). Un modèle peut ainsi apprendre *qui* signe plutôt que *ce qui* est jugé : c'est un biais à éliminer, d'autant que le droit français interdit le profilage des magistrats (section 9).

Validité du protocole. Le modèle final a été choisi sur la validation, le test n'a servi qu'une fois, et les comparaisons à

données égales neutralisent l'avantage de taille de A et B. Deux réserves : un seul tirage aléatoire (pas d'intervalle de confiance), et un test de généralisation temporelle limité à 95 arrêts.

9 Limites et perspectives

9.1 Limites

- **LEGAL-BERT non utilisé.** LEGAL-BERT [4] est pré-entraîné sur des textes juridiques *anglais*. L'appliquer à nos arrêts imposerait de les traduire d'abord en anglais : cela ajoute un second modèle lourd et lent, et la traduction déforme un vocabulaire sans équivalent exact en droit anglo-saxon (« cassation », « pourvoi », « dispositif »). La comparaison mesurerait alors « traduction + modèle » et non le modèle lui-même. Nous avons donc retenu CamemBERT, pré-entraîné sur du français.
- **Puissance de calcul.** CamemBERT n'a vu que 20 000 exemples, une seule époque et 256 tokens ; un entraînement complet (80 000 exemples, plusieurs époques) demanderait un GPU dédié.
- **Découpage aléatoire.** Le test principal mélange toutes les époques ; seul le test sur les Bulletins 2026 mesure la généralisation dans le temps, sur 95 arrêts seulement.
- **Étiquettes imparfaites.** Les zones et issues de Judilibre sont en partie produites automatiquement ; la classe « Autre » regroupe des issues hétérogènes.
- **Règles fragiles.** Elles dépendent des formats d'écriture : un nouveau modèle d'en-tête ou un OCR imparfait suffit à les mettre en défaut, et les ordonnances sans en-tête daté restent mal couvertes.
- **Une seule juridiction.** Rien ne garantit que les modèles fonctionnent sur les cours d'appel ou sur d'autres pays.

9.2 Perspectives

- **Modèles juridiques et multilingues** : comparer à JuriBERT [13], ou à LEGAL-BERT après traduction automatique, pour mesurer l'apport d'un pré-entraînement juridique.
- **Textes longs** : Longformer [21] ou découpage du texte en fenêtres pour lire la décision entière.
- **Entités nommées** : annoter quelques centaines d'arrêts (juges, parties, avocats, juridictions) et entraîner un modèle de NER, à la place des règles fragiles.
- **Découpage temporel** : entraîner sur le passé et tester sur les années récentes.
- **Extension à la Tunisie** : la jurisprudence tunisienne étant publiée en arabe, il faudrait construire un corpus (extraction, nettoyage, annotation) et utiliser un modèle arabe comme AraBERT [22] ou un modèle multilingue.
- **Mise en production** : une interface web qui reçoit un PDF et affiche le JSON.

9.3 Éthique

Les décisions sont pseudonymisées par la Cour de cassation. En France, la loi interdit d'exploiter les données d'identité des magistrats pour évaluer ou prédire leurs pratiques (article 33 de la loi n° 2019-222 du 23 mars 2019).

L'apparition d'un nom de magistrat parmi les mots caractéristiques (section 8) montre qu'un système réel devrait retirer ces noms avant l'apprentissage.

10 Références

- [1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., Polosukhin, I. (2017). Attention Is All You Need. *NeurIPS*.
- [2] Devlin, J., Chang, M.-W., Lee, K., Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL-HLT*.
- [3] Martin, L., Muller, B., Ortiz Suárez, P. J., Dupont, Y., Romary, L., de la Clergerie, É., Seddah, D., Sagot, B. (2020). CamemBERT: a Tasty French Language Model. *ACL*.
- [4] Chalkidis, I., Fergadiotis, M., Malakasiotis, P., Aletras, N., Androutsopoulos, I. (2020). LEGAL-BERT: The Muppets straight out of Law School. *Findings of EMNLP*.
- [5] Chalkidis, I., Jana, A., Hartung, D., Bommarito, M., Androutsopoulos, I., Katz, D. M., Aletras, N. (2022). LexGLUE: A Benchmark Dataset for Legal Language Understanding in English. *ACL*.
- [6] Zhong, H., Xiao, C., Tu, C., Zhang, T., Liu, Z., Sun, M. (2020). How Does NLP Benefit Legal System: A Summary of Legal Artificial Intelligence. *ACL*.
- [7] Aletras, N., Tsarapatsanis, D., Preotjuc-Pietro, D., Lampos, V. (2016). Predicting judicial decisions of the European Court of Human Rights: an NLP perspective. *PeerJ Computer Science*.
- [8] Chalkidis, I., Androutsopoulos, I., Aletras, N. (2019). Neural Legal Judgment Prediction in English. *ACL*.
- [9] Şulea, O.-M., Zampieri, M., Vela, M., van Genabith, J. (2017). Predicting the Law Area and Decisions of French Supreme Court Cases. *RANLP*.
- [10] Medvedeva, M., Wieling, M., Vols, M. (2023). Rethinking the field of automatic prediction of court decisions. *Artificial Intelligence and Law*, 31, 195–212.
- [11] Kim, Y. (2014). Convolutional Neural Networks for Sentence Classification. *EMNLP*.
- [12] Salton, G., Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513–523.
- [13] Douka, S., Abdine, H., Vazirgiannis, M., El Hamdani, R., Restrepo Amariles, D. (2021). JuriBERT: A Masked-Language Model Adaptation for French Legal Text. *Natural Legal Language Processing Workshop*.
- [14] Leitner, E., Rehm, G., Moreno-Schneider, J. (2019). Fine-grained Named Entity Recognition in Legal Documents. *SEMANTiCS*.
- [15] Honnibal, M., Montani, I., Van Landeghem, S., Boyd, A. (2020). spaCy: Industrial-strength Natural Language Processing in Python. Zenodo.
- [16] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [17] Wolf, T. et al. (2020). Transformers: State-of-the-Art Natural Language Processing. *EMNLP: System Demonstrations*.
- [18] Jeannot, A. (2024). Jurisprudence : jeu de données des décisions de la Cour de cassation (Judilibre). Hugging Face, antoinejeannot/jurisprudence.
- [19] Cour de cassation. Judilibre, moteur de recherche et open data des décisions de justice. courdecassation.fr.
- [20] Smith, R. (2007). An Overview of the Tesseract OCR Engine. *ICDAR*.
- [21] Beltagy, I., Peters, M. E., Cohan, A. (2020). Longformer: The Long-Document Transformer. arXiv:2004.05150.
- [22] Antoun, W., Baly, F., Hajj, H. (2020). AraBERT: Transformer-based Model for Arabic Language Understanding. *OSACT Workshop*.

Annexe A Figures complémentaires

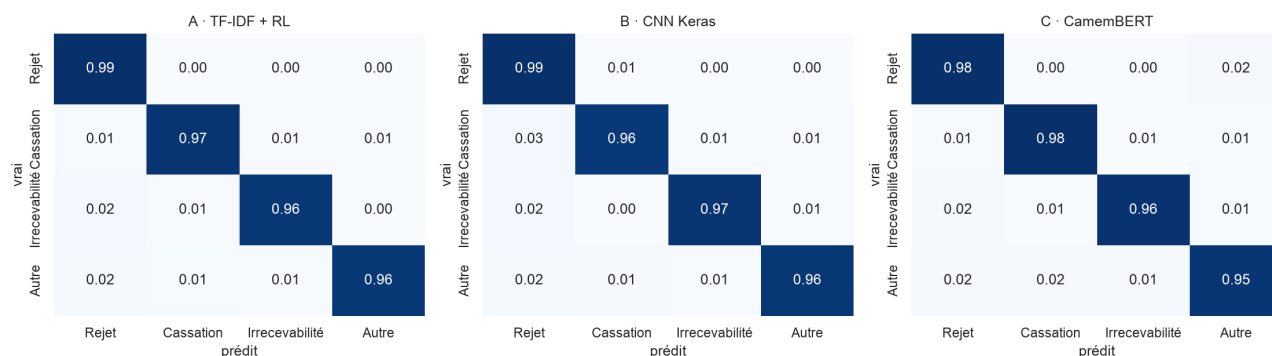


Figure 5 : Matrices de confusion normalisées par ligne (vraie classe) sur le test, pour les trois modèles. La diagonale donne le rappel de chaque classe.

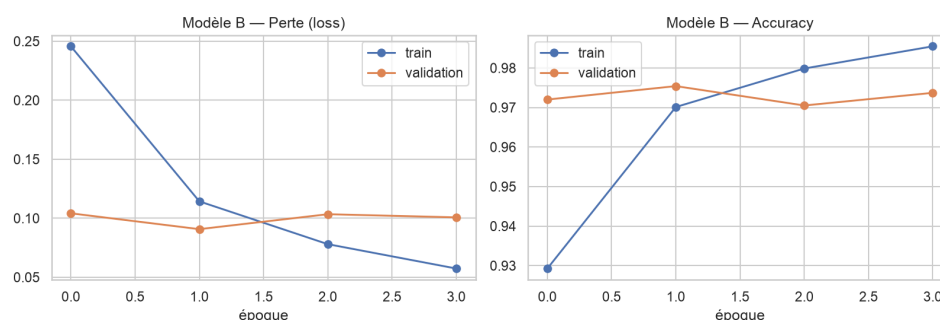


Figure 6 : Courbes d'apprentissage du modèle B (CNN Keras) : perte et accuracy sur l'entraînement et la validation. L'arrêt précoce garde les poids de la meilleure époque.

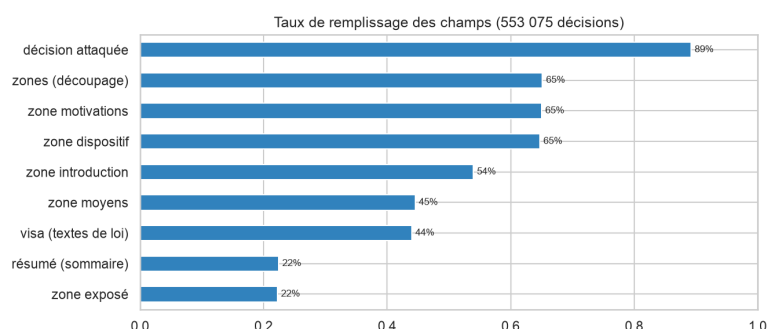


Figure 7 : Taux de remplissage des champs du jeu de données (553 075 décisions).

Annexe B Exemple de sortie JSON (extrait)

```
{
  "source_file": "Bulletin_criminel_2026-7.pdf",
  "decision_index": 1,
  "metadata": {
    "court": "Cour de cassation",
    "chamber": "Chambre criminelle",
    "date": "2026-07-29",
    "case_number": "26-83.088",
    "president": "M. Sottet"
  },
  "narrative": {
    "facts": "Faits et procédure 1. Il résulte de l'ordonnance attaquée et des pièces sou ...",
    "claims": "Examen du moyen Enoncé du moyen 6. Le moyen critique l'arrêt attaqué en ce ..."
  },
  "legal_foundations": {
    "statutes": [
      "article 803-8, ii, du code de procédure pénale",
      "article 6 de la convention européenne des droits de l'homme",
      "article 803-8 du code de procédure pénale"
    ],
    "previous_decision": {
      "court": "Cour d'appel de Nouméa",
      "date": "2026-04-30"
    }
  }
}
```

```

},
"outcome": {
  "verdict_rules": "Cassation",
  "verdict_header": "Cassation",
  "orders": [
    "CASSE et ANNULE, en toutes ses dispositions, l'ordonnance susvisée de ...",
    "RENVIOIE la cause et les parties devant la juridiction du président de ...",
    "DIT n'y avoir lieu à mise en liberté de M. [Q] ..."
  ],
  "reasoning": "Réponse de la Cour Sur le moyen, pris en ses première et deuxième branches ..."
},
"prediction": {
  "model": "A . spaCy + TF-IDF + régression logistique",
  "verdict": "Cassation",
  "confidence": 0.9902,
  "probabilities": {
    "Rejet": 0.0004,
    "Cassation": 0.9902,
    "Irrecevabilité": 0.0017,
    "Autre": 0.0042
  }
}
}
}

```